DOT-Domain
Comprehensive model reference

0.3.0 / 77 public types / generated structural reference and reviewed explanations

Read this alongside the single-page A4 model overview. This guide explains each building block, its JSON fields and its relationships. Java models and their generated schemas remain authoritative.

> **How to read the tables**
>
> Required means required by the base schema for the full standalone type. Conditional / optional does not mean always omittable: parent context and conditional rules can require a value. Skill and service Contract references have intentional ID-only views. Java constructor defaults do not waive raw JSON requirements. Defaults not declared by the schema are not inferred by this generator.
>
> Relationships come from the existing Java-derived Mermaid graph. Diagram multiplicities are association guidance; the field table, conditional schema and Java validation determine valid documents. An incoming relationship is not exclusive ownership.
>
> Java/schema checks establish declaration validity, not cross-DOT resolution, network reachability, authorization or live acceptance. Locks, resolved endpoints and runtime state belong to the CLI, not this portable model inventory.

Reading route

1. Identity and capabilities

   Dot / Metadata / Skill / Version

2. Contracts and Anchors

   Anchor / AnchorContract / AnchorDirection / AnchorRequirement / AnchorRole / Contract / ContractPayload / ContractStandard / DataFormat

3. Runtimes, deployment and configuration

   AnchorBinding / CdiDeviceRequirement / ConfigFile / Deployment / DeploymentConfig / DeploymentManifest / DeploymentManifestType / DeploymentService / DockerCompose / DockerImage / DockerImageTag / EnvFile / EnvironmentVariable / HelmChart / HelmValues / KubernetesManifest / OciLinuxNamespace / OciRuntimeRequirements / Platform / PlatformEngine / Runtime / ServicePort / Vagrantfile / VolumeMount / WebAssemblyExecution / WebAssemblyModule

4. Compositions and connections

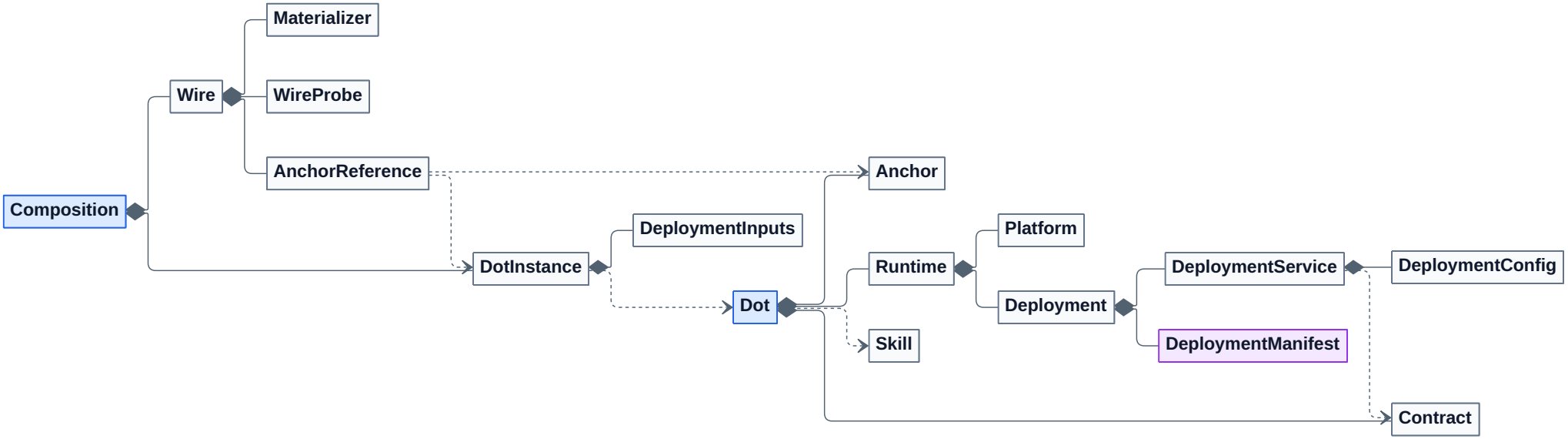
   AnchorReference / CdiDeviceBinding / ComposeWorkloadBinding / Composition / ContractInput / DeploymentConfigFile / DeploymentEnvironment / DeploymentInputs / DeploymentVolumeSource / DotInstance / ExposedAnchor / KubernetesWorkloadBinding / Materializer / MaterializerTarget / MaterializerType / MaterializerValueFrom / NetworkBinding / OciRuntimeGrant / OperationParameterBinding / SecretReference / SecurityBinding / SecurityParameterBinding / ServicePortReference / ServiceReference / ServiceWorkloadBinding / Wire / WireSecurity

5. Probes and assertions

   AsyncAPIProbe / AsyncAPIProbeAuthentication / AsyncAPIProtocol / GRPCProbe / GRPCResponseMatch / GraphQLOperation / GraphQLProbe / GraphQLTransport / JSONExpectation / OpenAPIProbe / WireProbe

Canonical Prometheus projections retain real declarations and pins. Other examples come from the existing reviewed Java compatibility corpus and illustrate individual value shapes only; they can use placeholder URLs, images or graph references and are not runnable product recommendations. ContractInput uses the current explicit-owner source convention.

Core relationships at a glance



Diamond: containment. Solid arrow: typed association. Dashed arrow: ID/name reference. Blue: aggregate roots. Purple: manifest base. This compact diagram omits supporting types; every public type and relationship is explained in the following chapters.

Identity and capabilities

Dot

Describe one reusable software product or component: its identity, capabilities, interfaces and supported deployment alternatives. A DOT definition is not a running instance or a complete multi-tool use case.

Keep one canonical product definition for the selected tool release. A composition selects instances and deployments, connects Anchors and supplies use-case inputs; do not copy product definitions for each scenario. The CLI deploys from the selected dot.json, not from nearby Dockerfiles or manifests. Java validation checks the aggregate and its declared relationships; catalogue resolution, verified Contract downloads and live acceptance are separate checks.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`id`	string	Required	Stable, case-sensitive component identity. A release-qualified identity must agree exactly with name and version. Legacy textual and UUID-shaped keys remain readable, but are not automatic aliases for a different catalogue entry. minimum length: 1; pattern restrictions (see schema)
`name`	string	Required	Readable product or component name, kept separate from the bundle directory and runtime-instance name. For a qualified identity, changing this name also changes the identity relationship that must be reviewed. minimum length: 1
`version`	Version	Required	Release of the represented software, not a revision counter for catalogue edits. Image packaging tags, Helm chart versions and Contract-language versions are separate. Label unreleased planning, source-adapted and test components honestly. pattern restrictions (see schema)
`description`	string	Conditional / optional	Optional human explanation of purpose, intended use and important limitations. It does not replace machine-readable Contracts, deployment inputs or acceptance evidence.
`groups`	array of string	Conditional / optional	Open classification labels rather than a closed domain enum. Ownership labels such as internal and external are interpreted by workflows; they do not grant permissions or establish production maturity.
`metadata`	Metadata	Required	Audit timestamps for this definition. Java construction provides initial values, but raw JSON still needs the required timestamp properties; metadata is not live deployment state.
`skills`	array of Skill (ID reference)	Required	Capability references serialized as ID-only objects. Choose at least one meaningful catalogue capability; its label and description belong to the Skill catalogue rather than being repeated in every DOT. minimum items: 1
`contracts`	array of (Contract (ID reference) or Contract)	Required	Authoritative interface declarations owned by this exact DOT release. Published catalogue roots need complete URL-backed declarations. References to dependency interfaces belong on Anchors with an explicit origin, not as copied declarations here.
`anchors`	array of Anchor	Conditional / optional	Embedded logical integration points provided or required by the component. Their IDs are local to this DOT and are the keys used by service bindings and composition wires; addresses and credentials do not belong here.

JSON field	Value	Base presence	Meaning / constraints
`runtimes`	array of Runtime	Required	Alternative ways the component can run on its supported platforms. A deployment instance chooses one named deployment; declaring several alternatives does not request that all of them be launched.

Relationships

- To Metadata: containment via metadata; diagram multiplicity 1.
- To Skill: ID/name reference via skills (ref); diagram multiplicity 1..*.
- To Version: containment via version; diagram multiplicity 1.
- To Anchor: containment via anchors; diagram multiplicity 0..*.
- To Contract: containment via contracts; diagram multiplicity 0..*.
- To Runtime: containment via runtimes; diagram multiplicity 0..*.
- From DotInstance: ID/name reference via dotId / source.

Canonical Prometheus projection


```
{
  "id": "Prometheus@3.15.1",
  "name": "Prometheus",
  "version": "3.15.1",
  "groups": [
    "external"
  ],
  "metadata": {
    "created": "2026-07-10T16:30:00Z",
    "updated": "2026-10-14T15:45:00Z"
  },
  "skills": [
    {
      "id": "Metrics-Scraping"
    }
  ],
  "contracts": [
    {
      "id": "Prometheus@3.15.1::OpenAPI::HTTP",
      "name": "HTTP",
      "version": "3.1.0",
      "standard": "OpenAPI",
      "format": "YAML",
      "payload": {
        "url":
"https://raw.githubusercontent.com/prometheus/prometheus/v3.15.1/web/api/v1/testdata/openapi_3.1_golden.yaml",
        "sha256": "406b517ed3800c107b3afbc2d9712f2e487269fe8c8b55a075dce46823d8d7c0"
      }
    }
  ],
  "anchors": [
    {
      "id": "build-information",
      "name": "build-information",
      "role": "PROVIDES",
      "contract": {
        "id": "Prometheus@3.15.1::OpenAPI::HTTP",
        "selector": "#/paths/~1status~1buildinfo/get"
      }
    }
  ],
  "runtimes": [
    {
      "platform": {
        "engine": "DOCKER",
        "version": "latest"
```

```
      },
      "deployments": [
        {
          "id": "docker-compose",
          "manifest": {
            "type": "DOCKER_COMPOSE",
            "format": "YAML",
            "payload": "services:\n  prometheus:\n    image:
docker.io/prom/prometheus:v3.15.1@sha256:fa3302279710be2f21a8301ec17a8b4f2301fd829bd113839b3b6b02b6f191a2\n
ports:\n      - \"${DOTX_HTTP_PORT:-18909}:9090\"\n    volumes:\n      - prometheus-data:/prometheus\nvolumes:\n
prometheus-data: {}\n"
          },
          "services": [
            {
              "id": "prometheus",
              "name": "Prometheus Server",
              "host": "prometheus",
              "ports": [
                {
                  "name": "http",
                  "protocol": "HTTP",
                  "port": 9090
                }
              ],
              "contracts": [
                {
                  "id": "Prometheus@3.15.1::OpenAPI::HTTP"
                }
              ],
              "anchorBindings": [
                {
                  "anchorId": "build-information",
                  "port": "http"
                }
              ],
              "config": {
                "environment": [
                  {
                    "name": "DOTX_HTTP_PORT",
                    "defaultValue": "18909",
                    "description": "Host port used by the Docker Compose example."
                  }
                ],
                "configFiles": [],
                "volumes": [
                  {
                    "name": "prometheus-data",
                    "path": "/prometheus",
                    "description": "Prometheus TSDB storage for the Compose mechanism."
                  }
                ]
              }
            }
          ]
        }
      ]
    }
  ]
}
```

Metadata

Record when a domain definition was created and last updated. These are audit values for the authored record, not workload start times or health observations.

Java constructors and builders initialize timestamps, but authors of raw JSON must supply the properties required by the generated schema. Callers manage subsequent updates; changing another field does not automatically refresh updated. The model does not enforce chronological ordering between these two timestamps or turn them into a version history.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`created`	string	Required	Instant at which this definition record was created, serialized as an offset-bearing date-time. Keep it distinct from the release date of the represented software and from a deployment creation time. format: date-time; pattern restrictions (see schema)
`updated`	string	Required	Instant recorded for the latest authored change. Update it deliberately when appropriate; it is not an automatically maintained clock, a lock digest or proof that a live instance matches the definition. format: date-time; pattern restrictions (see schema)

Relationships

- From Dot: containment via metadata; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "created": "2026-10-26T00:00:00Z",
  "updated": "2026-10-26T00:00:00Z"
}
```

Skill

Name a reusable, high-level capability so that components can be classified and discovered by what they do. A Skill is not executable code, an API operation or a deployment recipe.

A standalone Skill has its own descriptive metadata, while Dot.skills serializes only its identity. The catalogue resolves that identity to the authoritative label, description and capability category. Domain reference validation checks the key; repository validation checks that the referenced capability exists and is not merely a category asset.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`description`	string	Conditional / optional	Optional explanation of the capability and its scope. Describe the ability without duplicating the interface Contract or claiming that a runtime has been verified.
`id`	string	Required	Stable, case-sensitive capability key. Moving a catalogue file or changing a display label must not silently create a different identity; unknown keys are not interpreted as names. minimum length: 1; pattern restrictions (see schema)
`name`	string	Required	Human-readable capability label for a complete Skill entity. It is omitted from the ID-only reference view inside a DOT. minimum length: 1

Relationships

- From Dot: ID/name reference via skills (ref); diagram multiplicity 1..*.

Java compatibility shape example - not runnable

```
{
  "id": "Metrics-Scraping",
  "name": "Metrics scraping"
}
```

Version

Carry a semantic software version or the special latest selector as a single JSON string. Use it where the owning model needs a software or platform version rather than a free-form label.

The Java default is 0.0.0; it is not evidence of a released component. Concrete values follow SemVer precedence, including prereleases; build metadata is ignored for precedence comparisons. The latest selector sorts after concrete versions and delegates selection to its context, so it is not an immutable artifact pin. Contract.version is a separate document-language string, not this value object. There is no nested JSON value property.

Java authority / Generated JSON Schema

JSON representation: string. pattern restrictions (see schema)

Relationships

- From Dot: containment via version; diagram multiplicity 1.
- From HelmChart: containment via version; diagram multiplicity 1.
- From Platform: containment via version; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
"0.0.0"
```

Contracts and Anchors

Anchor

Give one provided or required interaction a stable, DOT-local integration key. Compositions connect these logical points; deployment service bindings connect locally provided, non-brokered interfaces to named ports.

Choose exactly one communication mode: a Contract operation, a supported provider-neutral requirement, or explicit contractless communication facts. Do not duplicate direction, protocols or media types for a Contract-backed Anchor. Aggregate validation checks references, selector syntax and deployment bindings; the CLI resolves actual Contract operations. Declarations, optional semantic labels and a compatible wire are not live reachability, authentication or production acceptance.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`id`	string	Required	Stable key used by composition endpoints and deployment Anchor bindings. Prefer meaningful DOT-local kebab-case names. Legacy UUID strings remain readable, but display names and selectors are not aliases for this key. minimum length: 1; pattern restrictions (see schema); conditional constraints (see schema)
`name`	string	Required	Readable label for the interaction. It can help an author choose an Anchor, but wires and service bindings reference id rather than this display label. minimum length: 1
`role`	AnchorRole	Required	Whether this component offers the interaction or depends on it. This logical provider/consumer relationship is independent of which participant initiates a network request or the direction of payload flow. allowed: "PROVIDES", "REQUIRES"
`semanticTypes`	array of string	Conditional / optional	Optional defined vocabulary identifiers for domain meaning, not API documentation URLs or payload-schema pointers. The CLI treats them as opaque URI strings: nonempty lists on both sides must intersect exactly; an absent list imposes no restriction and supplies no semantic proof. It performs no ontology inference.
`contract`	AnchorContract	Conditional / optional	Reference to an authoritative Contract plus one standard-specific operation selector. Local ownership resolves through this DOT; external ownership requires the exact originating DOT source. Communication facts come from the selected document.
`requirement`	AnchorRequirement	Conditional / optional	Provider-neutral, versioned requirement on a REQUIRES Anchor. A composition chooses and verifies the concrete provider. It cannot be combined with a Contract reference or explicit contractless communication facts. exact value "OIDC_DISCOVERY_1_0"
`direction`	AnchorDirection	Conditional / optional	Payload-flow direction relative to the declaring DOT, only for a contractless Anchor. Do not infer it solely from PROVIDES or REQUIRES; request/response interactions and transport initiation are different concepts. allowed: "INGRESS", "EGRESS", "BIDIRECTIONAL"
`protocols`	array of string	Conditional / optional	Explicit communication protocols for a contractless interaction. Declare the actual supported boundary rather than treating a semantic label, file format or service name as a protocol.

JSON field	Value	Base presence	Meaning / constraints
`mediaTypes`	array of string	Conditional / optional	Explicit payload media types for a contractless interaction. These describe the exchanged representation, not a full payload schema or a guarantee that another component understands its domain meaning.

Relationships

- From Dot: containment via anchors; diagram multiplicity 0..*.
- To AnchorContract: containment via contract; diagram multiplicity 0..1.
- To AnchorDirection: typed association via direction; diagram multiplicity 0..1.
- To AnchorRequirement: typed association via requirement; diagram multiplicity 0..1.
- To AnchorRole: typed association via role; diagram multiplicity 1.
- From AnchorReference: ID/name reference via anchorId.
- From AnchorBinding: ID/name reference via anchorId.

Canonical Prometheus projection

```
{
  "id": "build-information",
  "name": "build-information",
  "role": "PROVIDES",
  "contract": {
    "id": "Prometheus@3.15.1::OpenAPI::HTTP",
    "selector": "#/paths/~1status~1buildinfo/get"
  }
}
```

AnchorContract

Select one composable interaction from an existing authoritative Contract. Use it on an Anchor rather than copying an interface document or declaring another product-owned Contract.

The referenced identity, origin and selector must agree. OpenAPI and AsyncAPI use operation references; Protobuf uses a service method and GraphQL uses a root-field coordinate. A data-schema pointer is not an operation. Java aggregate validation checks local relationships and selector syntax; the CLI resolves the exact originating declaration, verifies its bytes and checks the selected operation. Matching shapes in independently authored Contracts do not make their identities interchangeable.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`id`	string	Required	Exact, case-sensitive Contract identity. An external dependency retains the producer identity; merely declaring a syntactically valid key does not make a Contract available. minimum length: 1; pattern restrictions (see schema)
`selector`	string	Required	Standard-specific operation reference within the authoritative document. Select the interaction rather than a request/response data type; required operation parameters and security still need composition and deployment inputs. minimum length: 1
`source`	string	Conditional / optional	Exact originating DOT coordinate for an external Contract reference in the selected catalogue. Omit it for a locally owned declaration. An unavailable origin is an error, not permission to search sibling folders, infer another release or copy a dependency declaration. pattern restrictions (see schema)

Relationships

- From Anchor: containment via contract; diagram multiplicity 0..1.
- To Contract: ID/name reference via id.

Java compatibility shape example - not runnable

```
{
  "id": "Example@1.0.0::OpenAPI::HTTP",
  "selector": "#/paths/~1ready/get"
}
```

AnchorDirection

Describe the direction of data flow at a contractless interaction relative to the declaring DOT. It fills in a communication fact that cannot be derived from an authoritative Contract.

Ingress means flow into the DOT, egress means flow out, and bidirectional means both. Keep this separate from the Anchor provider/consumer role and from the direction of a TCP connection. The current contractless matcher requires complementary directions alongside compatible protocols and media types; the enum alone proves none of those other conditions.

Java authority / Generated JSON Schema

Exact JSON values: `INGRESS`, `EGRESS`, `BIDIRECTIONAL`.

JSON representation: string. allowed: "INGRESS", "EGRESS", "BIDIRECTIONAL"

Relationships

- From Anchor: typed association via direction; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
"INGRESS"
```

AnchorRequirement

State a narrow, versioned provider-neutral interface requirement instead of hard-coding a particular producer Contract. The current supported profile concerns OIDC discovery.

Use it only on a REQUIRES Anchor and do not combine it with Contract or explicit contractless fields. A composition wire selects the real provider and the CLI checks the supported requirement profile against its verified Contract. This is not an arbitrary semantic label, automatic provider discovery or full OIDC, TLS and production IAM acceptance.

Java authority / Generated JSON Schema

Exact JSON values: `OIDC_DISCOVERY_1_0`.

JSON representation: string. exact value "OIDC_DISCOVERY_1_0"

Relationships

- From Anchor: typed association via requirement; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
"OIDC_DISCOVERY_1_0"
```

AnchorRole

Distinguish a component that provides an interaction from one that requires it. This is the logical relationship used to connect a provider Anchor to a consumer Anchor.

Role is not payload-flow direction, server/client process type or permission to call an API. A component can provide an HTTP operation even when another component initiates the request. Complementary roles do not replace Contract identity, operation, protocol, security or live acceptance checks.

Java authority / Generated JSON Schema

Exact JSON values: `PROVIDES`, `REQUIRES`.

JSON representation: string. allowed: "PROVIDES", "REQUIRES"

Relationships

- From Anchor: typed association via role; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
"PROVIDES"
```

Contract

Identify one authoritative interface specification and the exact downloadable document that describes it. Anchors select interactions from that document instead of copying its protocol and payload details.

Keep the declaration at its owning DOT and retain that identity when another component consumes or implements the interface. Qualified identities must agree with the exact owning DOT release, standard and local Contract name. Complete definitions carry a verified URL/hash payload; an ID-only reference view is not a publishable owned declaration. Domain validation checks declaration relationships and metadata, while the CLI fetches, hashes and parses the document and resolves operations. Neither establishes that a producer implements the interface at runtime.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`format`	DataFormat	Required	Encoding of the downloaded document, not its transport protocol. It must be supported by the chosen standard; an extension or a matching file name does not establish compatibility. allowed: "YAML", "JSON", "PROTOBUF", "GRAPHQL"
`id`	string	Required	Stable interface identity, independent of its download URL. Catalogue identities include the originating DOT release and the exact standard/name suffix. Consumers retain this key rather than minting a consumer-owned copy. minimum length: 1; pattern restrictions (see schema)
`name`	string	Required	Local, case-sensitive interface name within the owning DOT. It is part of the qualified identity, not a hostname, an operation selector or the upstream API release number. minimum length: 1
`payload`	ContractPayload	Required	Structured raw-document location and integrity pin. The specification bytes are downloaded separately; this property does not embed the document or provide a local Contract-folder fallback.

JSON field	Value	Base presence	Meaning / constraints
`standard`	ContractStandard	Required	Interface language used to interpret the document and Anchor selectors. Use the exact generated wire spelling; the Java enum constant name is not necessarily the serialized value. allowed: "OpenAPI", "AsyncAPI", "Protobuf", "GraphQL"
`version`	string	Conditional / optional	Version of the document language declared by the verified payload: the OpenAPI or AsyncAPI marker, or the native Protobuf syntax/edition. Do not copy a tool release or info.version. GraphQL SDL has no such marker and omits this property.

Relationships

- From Dot: containment via contracts; diagram multiplicity 0..*.
- To DataFormat: typed association via format; diagram multiplicity 1.
- To ContractPayload: containment via payload; diagram multiplicity 0..1.
- To ContractStandard: typed association via standard; diagram multiplicity 1.
- From DeploymentService: ID/name reference via contracts (ref); diagram multiplicity 0..*.
- From AnchorContract: ID/name reference via id.
- From ContractInput: ID/name reference via id.

Canonical Prometheus projection


```
{
  "id": "Prometheus@3.15.1::OpenAPI::HTTP",
  "name": "HTTP",
  "version": "3.1.0",
  "standard": "OpenAPI",
  "format": "YAML",
  "payload": {
    "url":
"https://raw.githubusercontent.com/prometheus/prometheus/v3.15.1/web/api/v1/testdata/openapi_3.1_golden.yaml",
    "sha256": "406b517ed3800c107b3afbc2d9712f2e487269fe8c8b55a075dce46823d8d7c0"
  }
}
```


ContractPayload

Locate the raw bytes of a Contract specification and pin their integrity. Use this inside a complete Contract declaration, not as the address of a deployed API.

The URL must be absolute HTTPS without embedded credentials or a fragment. Prefer immutable raw-document locations; the CLI verifies the digest before interpreting the bytes and rechecks cached content. A correct hash establishes byte identity, not vendor authority, license permission, producer conformance or live availability. Keep specification provenance and acceptance limitations explicit.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`sha256`	string	Required	Lower-case SHA-256 of the exact downloaded bytes. Formatting or newline changes alter this pin even if a reader considers the document equivalent; review changed bytes rather than silently replacing the expected digest. minimum length: 1; pattern restrictions (see schema)

JSON field	Value	Base presence	Meaning / constraints
`url`	string	Required	Download location for the raw specification, not an HTML documentation viewer, endpoint base URL or operation fragment. Runtime addresses and credentials are deployment-instance concerns. format: uri; pattern restrictions (see schema)

Relationships

- From Contract: containment via payload; diagram multiplicity 0..1.

Canonical Prometheus projection

```
{
  "url":
"https://raw.githubusercontent.com/prometheus/prometheus/v3.15.1/web/api/v1/testdata/openapi_3.1_golden.yaml",
  "sha256": "406b517ed3800c107b3afbc2d9712f2e487269fe8c8b55a075dce46823d8d7c0"
}
```

ContractStandard

Choose the interface-description language used by a Contract. This choice determines how the document is parsed, which encoding is valid and how an Anchor operation is selected.

Use the exact serialized values shown by the generated enum reference; alternative capitalization, whitespace and Java constant names are not aliases. Standard version and document format remain separate properties. Selecting a supported language is not a claim that every optional feature or live protocol behavior is implemented by the current CLI.

Java authority / Generated JSON Schema

Exact JSON values: `OpenAPI`, `AsyncAPI`, `Protobuf`, `GraphQL`.

JSON representation: string. allowed: "OpenAPI", "AsyncAPI", "Protobuf", "GraphQL"

Relationships

- From Contract: typed association via standard; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
"OpenAPI"
```

DataFormat

Identify the serialization format of a document or manifest payload. It describes how bytes are represented, not the purpose of the interface or the infrastructure that runs it.

The owning field determines which formats are valid. Contract formats must match their Contract standard, and Docker Compose, Kubernetes manifests and Helm values accept only their supported structured text encodings. A format enum does not add a new deployment mechanism, turn a Vagrantfile into YAML or describe an HTTP media type.

Java authority / Generated JSON Schema

Exact JSON values: `YAML`, `JSON`, `PROTOBUF`, `GRAPHQL`.

JSON representation: string. allowed: "YAML", "JSON", "PROTOBUF", "GRAPHQL"

Relationships

- From Contract: typed association via format; diagram multiplicity 1.
- From DockerCompose: typed association via format; diagram multiplicity 1.

- From HelmValues: typed association via format; diagram multiplicity 1.
- From KubernetesManifest: typed association via format; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
"YAML"
```

Runtimes, deployment and configuration

AnchorBinding

Maps a logical DOT Anchor to a named port of one deployment service.

Use an exact Anchor ID from the same DOT and an existing port name on the containing service. The binding connects logical interface selection to deployment structure; it does not contain a URL, credentials or a host-port publication rule.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`anchorId`	string	Required	The exact DOT-local Anchor identifier being materialized. minimum length: 1; pattern restrictions (see schema); conditional constraints (see schema)
`port`	string	Required	The name of the containing service port, not its numeric port value. minimum length: 1

Relationships

- From DeploymentService: containment via anchorBindings; diagram multiplicity 0..*.
- To Anchor: ID/name reference via anchorId.
- To ServicePort: ID/name reference via port.

Canonical Prometheus projection

```
{
  "anchorId": "build-information",
  "port": "http"
}
```

CdiDeviceRequirement

Declares a service's need for a device exposed through the Container Device Interface.

The DOT names a logical requirement and device kind, not a host device path or a concrete allocated device. Deployment inputs bind a fully qualified device of the matching kind. Device allocation, CDI specification installation and authorization remain operator or orchestrator responsibilities.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`description`	string	Conditional / optional	An explanation of why the service needs this device.
`kind`	string	Required	The CDI vendor/class identity that a supplied device must match. minimum length: 1; maximum length: 317; pattern restrictions (see schema)
`name`	string	Required	The service-local logical identifier used when binding a concrete device. minimum length: 1; maximum length: 63; pattern restrictions (see schema)

Relationships

- From DeploymentConfig: containment via cdiDevices; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "name": "gpu",
  "kind": "vendor.example/gpu"
}
```

ConfigFile

Documents a configuration file expected by a service.

The path is the destination inside the workload, not a source file on the operator's machine. The declaration does not create a mount or generated file. Instance inputs provide replacement content; sensitive files must not embed reusable default content.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`contentType`	string	Conditional / optional	The media type describing how the file content should be interpreted.
`defaultContent`	string	Conditional / optional	Reusable non-sensitive content available when no instance replacement is supplied.
`description`	string	Conditional / optional	An explanation of the file's role.
`path`	string	Required	The destination path where the workload expects the configuration file. minimum length: 1
`required`	boolean	Conditional / optional	Whether content must be supplied when the declaration provides none.
`sensitive`	boolean	Conditional / optional	Whether tooling must protect the file content as sensitive information.

Relationships

- From DeploymentConfig: containment via configFiles; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "path": "/etc/example/config.yaml",
  "defaultContent": "enabled: true"
}
```

Deployment

Describes one selectable deployment mechanism and the logical services it materializes.

The manifest is the complete deployment input, not a pointer to arbitrary files beside dot.json. Keep service IDs distinct within the deployment and select the deployment explicitly. Configuration belongs to each service, while instance names, live state and lifecycle ownership belong to the executor.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`id`	string	Required	The stable name used to select this deployment alternative. minimum length: 1
`manifest`	DockerImage or DockerCompose or KubernetesManifest or HelmChart or Vagrantfile or WebAssemblyModule	Required	The platform-specific specification used to create the workload. conditional constraints (see schema)
`services`	array of DeploymentService	Required	The logical services supplied by this particular deployment.

Relationships

- To DeploymentService: containment via services; diagram multiplicity 0..*.
- To DeploymentManifest: containment via manifest; diagram multiplicity 1.
- From Runtime: containment via deployments; diagram multiplicity 1..*.

Java compatibility shape example - not runnable

```
{
  "id": "image",
  "manifest": {
    "type": "DOCKER_IMAGE",
    "image": "docker.io/example/api:1.0.0"
  },
  "services": []
}
```

DeploymentConfig

Declares the configuration options and execution requirements supported by one service.

This is not a bag of applied operator values. Keep environment names, configuration paths, volume names and paths, and CDI requirement names unambiguous within the service; separate services can reuse those keys. Instance inputs supply overrides, storage bindings and secrets. Requirements never grant privileges by themselves.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`cdiDevices`	array of CdiDeviceRequirement	Conditional / optional	Logical requirements for devices supplied through CDI.
`configFiles`	array of ConfigFile	Required	Configuration files the service can consume, with any reusable non-sensitive content.
`environment`	array of EnvironmentVariable	Required	Environment options understood by the service.
`ociRuntime`	OciRuntimeRequirements	Conditional / optional	Additional container capabilities or host-namespace requirements needing explicit authorization.
`volumes`	array of VolumeMount	Required	Logical mount points expected by the service.

Relationships

- From DeploymentService: containment via config; diagram multiplicity 1.
- To ConfigFile: containment via configFiles; diagram multiplicity 0..*.
- To EnvironmentVariable: containment via environment; diagram multiplicity 0..*.

- To VolumeMount: containment via volumes; diagram multiplicity 0..*.
- To CdiDeviceRequirement: containment via cdiDevices; diagram multiplicity 0..*.
- To OciRuntimeRequirements: containment via ociRuntime; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "environment": [],
  "configFiles": [],
  "volumes": [],
  "cdiDevices": []
}
```

DeploymentManifest

Provides the common base for concrete deployment specifications.

The existing type property selects the concrete manifest shape; it is not a second infrastructure selector. Supply the fields of that concrete manifest. Domain validation does not execute a payload, fetch a chart or establish deployment success.

Java authority / Generated JSON Schema

Manifest alternatives: DockerImage, DockerCompose, KubernetesManifest, HelmChart, Vagrantfile, WebAssemblyModule.

JSON field	Value	Base presence	Meaning / constraints
`type`	DeploymentManifestType	Required	The discriminator identifying how this manifest is deployed. conditional constraints (see schema)

Relationships

- To DockerCompose: inheritance.
- To DockerImage: inheritance.
- To HelmChart: inheritance.
- To KubernetesManifest: inheritance.
- To Vagrantfile: inheritance.
- To WebAssemblyModule: inheritance.
- From Deployment: containment via manifest; diagram multiplicity 1.
- To DeploymentManifestType: typed association via type; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "type": "DOCKER_IMAGE",
  "image": "docker.io/example/api:1.0.0"
}
```

DeploymentManifestType

Names how a workload is deployed and links that mechanism to its target platform.

Do not substitute this value for Platform.engine. The manifest subtype and its type must agree, and its target platform must match the enclosing runtime. Executor capabilities and installed tools remain separate from this model vocabulary.

Java authority / Generated JSON Schema

Exact JSON values: `DOCKER_IMAGE`, `DOCKER_COMPOSE`, `KUBERNETES_MANIFEST`, `HELM_CHART`, `VAGRANTFILE`, `WASM_MODULE`.

JSON representation: string. allowed: "DOCKER_IMAGE", "DOCKER_COMPOSE", "KUBERNETES_MANIFEST", "HELM_CHART", "VAGRANTFILE", "WASM_MODULE"

Relationships

- To PlatformEngine: typed association via platform; diagram multiplicity 0..1.
- From DeploymentManifest: typed association via type; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
"DOCKER_IMAGE"
```

DeploymentService

Describes a logical service provided by one deployment, including its interfaces and configuration surface.

Use stable service IDs for composition inputs and named ports for Anchor bindings. Contract entries are identity references, not copied declarations; their origin must resolve through the DOT interfaces. A logical service is not a native workload selector, and its host is not a promised public endpoint.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`id`	string	Required	The stable service identifier within its deployment. minimum length: 1
`name`	string	Required	The readable name of the service. minimum length: 1
`host`	string	Required	The host by which the service is reached inside the deployment. minimum length: 1
`ports`	array of ServicePort	Required	Named service ports that can be selected by Anchor bindings.
`contracts`	array of Contract (ID reference)	Required	ID-only references to the interfaces exposed or consumed by this service.
`anchorBindings`	array of AnchorBinding	Conditional / optional	Mappings from this DOT's logical Anchors to the service's named ports.
`config`	DeploymentConfig	Required	The environment, files, storage and execution requirements supported by this service.

Relationships

- To Contract: ID/name reference via contracts (ref); diagram multiplicity 0..*.
- To AnchorBinding: containment via anchorBindings; diagram multiplicity 0..*.
- To ServicePort: containment via ports; diagram multiplicity 0..*.
- To DeploymentConfig: containment via config; diagram multiplicity 1.
- From Deployment: containment via services; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "id": "api",
  "name": "API",
  "host": "api",
  "ports": [],
  "contracts": [],
  "config": {
    "environment": [],
    "configFiles": [],
    "volumes": [],
    "cdiDevices": []
  }
}
```

DockerCompose

Embeds a complete Docker Compose document for a multi-container deployment.

The payload, not a compose file beside dot.json, defines the deployment. Its declared format must be supported for Compose documents. Ordered environment-file payloads follow Compose precedence; instance overrides remain separate. Logical services and any native workload selection must agree with the selected deployment.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`type`	DeploymentManifestType	Required	The discriminator selecting the Docker Compose manifest. exact value "DOCKER_COMPOSE"
`format`	DataFormat	Required	The serialization format of the embedded Compose document. allowed: "YAML", "JSON", "PROTOBUF", "GRAPHQL"
`payload`	string	Required	The complete Compose document content. minimum length: 1
`envFiles`	array of EnvFile	Conditional / optional	Ordered environment-file payloads applied to the Compose invocation.

Relationships

- From DeploymentManifest: inheritance.
- To DataFormat: typed association via format; diagram multiplicity 1.
- To EnvFile: containment via envFiles; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "type": "DOCKER_COMPOSE",
  "format": "YAML",
  "payload": "services: {}"
}
```

DockerImage

Runs a pre-built container image as a Docker deployment alternative.

Reference a published image rather than expecting the CLI to build a nearby Dockerfile. Ordered environment-file payloads provide reusable defaults; later files and explicit instance environment values take precedence. Expose user-facing configuration through each logical service rather than adding deployment-level overrides to the model.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`type`	DeploymentManifestType	Required	The discriminator selecting the Docker image manifest. exact value "DOCKER_IMAGE"
`image`	DockerImageTag	Required	The compact image reference to pull and run.
`envFiles`	array of EnvFile	Conditional / optional	Ordered Docker-compatible environment-file payloads for reusable manifest defaults.

Relationships

- From DeploymentManifest: inheritance.
- To DockerImageTag: containment via image; diagram multiplicity 1.
- To EnvFile: containment via envFiles; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "type": "DOCKER_IMAGE",
  "image": "docker.io/example/api:1.0.0"
}
```

DockerImageTag

Represents a container image tag as a compact JSON string.

Although Java exposes registry, image and version components, JSON uses one image-reference string, not an object containing those fields. The Java parser is tag-based and rejects digest references. Image-reference parsing alone does not verify registry availability, artifact integrity or executor support.

Java authority / Generated JSON Schema

JSON representation: string.

Relationships

- From DockerImage: containment via image; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
"docker.io/example/api:1.0.0"
```

EnvFile

Stores reusable Docker-compatible environment-file content inside a manifest.

Use line-oriented dotenv content rather than a path to an adjacent file. The containing manifest determines ordering and override precedence. Keep deployment-instance values and credentials out of this reusable payload; EnvFile is not a secret reference.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`payload`	string	Required	The environment-file content passed to the Docker or Compose invocation. minimum length: 1

Relationships

- From DockerCompose: containment via envFiles; diagram multiplicity 0..*.
- From DockerImage: containment via envFiles; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "payload": "MODE=demo"
}
```

EnvironmentVariable

Documents an environment option accepted by a service.

The declaration explains the option; deployment inputs provide the applied value. A reusable default can be used when there is no override, but a sensitive option must not embed a default. Marking a value sensitive is not encryption or a substitute for secret provisioning.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`defaultValue`	string	Conditional / optional	Reusable non-sensitive text available when no instance override is supplied.
`description`	string	Conditional / optional	An explanation of what the option controls.
`name`	string	Required	The environment variable name consumed by the workload. minimum length: 1
`required`	boolean	Conditional / optional	Whether the deployer must supply a value when the declaration provides none.
`sensitive`	boolean	Conditional / optional	Whether tooling must treat values for this option as sensitive.

Relationships

- From DeploymentConfig: containment via environment; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "name": "MODE",
  "defaultValue": "demo"
}
```

HelmChart

References a Helm chart and preserves its reusable chart-level values.

Keep chart version separate from the represented tool release. The chart reference can use a repository or a self-contained OCI reference; an environment alias needs to be available where deployment runs. Values are ordered, with later entries overriding earlier ones. Release name and namespace are instance concerns, while service-facing configuration belongs on DeploymentService.config.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`chart`	string	Required	The chart name or self-contained chart reference. minimum length: 1
`repository`	string	Conditional / optional	The chart repository URL when the chart reference needs one. pattern restrictions (see schema)
`type`	DeploymentManifestType	Required	The discriminator selecting a Helm chart deployment. exact value "HELM_CHART"
`values`	array of HelmValues	Conditional / optional	Ordered Helm values documents merged for chart rendering and installation.
`version`	Version	Required	The chart version selector, not the application's software version. pattern restrictions (see schema)

Relationships

- From DeploymentManifest: inheritance.
- To Version: containment via version; diagram multiplicity 1.
- To HelmValues: containment via values; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "type": "HELM_CHART",
  "chart": "example/api",
  "version": "1.0.0",
  "values": [
    {
      "format": "YAML",
      "payload": "replicas: 1"
    }
  ]
}
```

HelmValues

Carries one values document for a Helm chart's template configuration.

Use this for chart-level settings such as resource configuration or persistence, rather than inventing extra portable model fields. The document format must be supported by Helm values validation. The containing chart controls merge order; supported environment, file and mount options still belong in service configuration.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`format`	DataFormat	Required	The serialization format of this values document. allowed: "YAML", "JSON", "PROTOBUF", "GRAPHQL"
`payload`	string	Required	The chart values content, equivalent to one values-file input. minimum length: 1

Relationships

- From HelmChart: containment via values; diagram multiplicity 0..*.
- To DataFormat: typed association via format; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "format": "YAML",
  "payload": "replicas: 1"
}
```

KubernetesManifest

Embeds the Kubernetes resource documents used by a deployment.

The payload is the reusable deployment authority, not an instruction to scan adjacent manifest files. Native resource syntax, tool readiness and live rollout are executor concerns beyond the domain's format checks. Do not confuse a logical service with the exact workload that an instance must select.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`format`	DataFormat	Required	The serialization format of the embedded Kubernetes documents. allowed: "YAML", "JSON", "PROTOBUF", "GRAPHQL"
`payload`	string	Required	The Kubernetes resource document content applied by the adapter. minimum length: 1
`type`	DeploymentManifestType	Required	The discriminator selecting the Kubernetes manifest deployment. exact value "KUBERNETES_MANIFEST"

Relationships

- From DeploymentManifest: inheritance.
- To DataFormat: typed association via format; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "type": "KUBERNETES_MANIFEST",
  "format": "YAML",
  "payload": "apiVersion: v1"
}
```

OciLinuxNamespace

Identifies a Linux namespace type in OCI execution requirements.

In hostNamespaces, this value requests sharing that namespace with the host, not creating a new isolated namespace. The serialized spelling follows OCI vocabulary. A model value is not permission to grant access, nor a guarantee of support by every adapter.

Java authority / Generated JSON Schema

Exact JSON values: `pid`, `network`, `mount`, `ipc`, `uts`, `user`, `cgroup`, `time`.

JSON representation: string. allowed: "pid", "network", "mount", "ipc", "uts", "user", "cgroup", "time"

Relationships

- From OciRuntimeGrant: typed association via hostNamespaces; diagram multiplicity 0..*.
- From OciRuntimeRequirements: typed association via hostNamespaces; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
"pid"
```

OciRuntimeRequirements

Declares additional Linux-container execution requirements using OCI vocabulary.

These are requests, not grants or unrestricted privileged mode. Compositions must explicitly authorize the declared set, and an adapter must fail closed when it cannot materialize the request. Duplicate declarations are rejected rather than silently removed during collection conversion.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`capabilities`	array of string	Required	Additional Linux process capabilities needed by the workload. distinct items
`hostNamespaces`	array of OciLinuxNamespace	Required	Namespace types that the workload needs to share with the host instead of isolating. distinct items

Relationships

- From DeploymentConfig: containment via ociRuntime; diagram multiplicity 0..1.
- To OciLinuxNamespace: typed association via hostNamespaces; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "capabilities": [
    "CAP_NET_RAW",
    "CAP_NET_ADMIN"
  ],
  "hostNamespaces": [
    "network",
    "pid"
  ]
}
```

Platform

Identifies the infrastructure targeted by a runtime, independently of its deployment mechanism.

Keep the platform version separate from the software release and chart version. Java construction defaults the engine to Kubernetes and the platform version to latest; those defaults do not waive required raw JSON properties. A platform declaration does not install the infrastructure or prove that its tools are available; the deployment environment resolves version selectors and checks readiness.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`engine`	PlatformEngine	Required	The infrastructure engine that hosts the workload. allowed: "DOCKER", "KUBERNETES", "VAGRANT", "WEBASSEMBLY"
`version`	Version	Required	The platform or runtime version selector, not the deployed product version. pattern restrictions (see schema)

Relationships

- From Runtime: containment via platform; diagram multiplicity 1.
- To Version: containment via version; diagram multiplicity 1.
- To PlatformEngine: typed association via engine; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "engine": "DOCKER",
  "version": "latest"
}
```

PlatformEngine

Names where a workload runs.

Use this infrastructure vocabulary on Platform. It is different from the manifest mechanism: multiple deployment mechanisms can target the same engine. An engine value does not promise that the current executor supports every operation on that platform.

Java authority / Generated JSON Schema

Exact JSON values: `DOCKER`, `KUBERNETES`, `VAGRANT`, `WEBASSEMBLY`.

JSON representation: string. allowed: "DOCKER", "KUBERNETES", "VAGRANT", "WEBASSEMBLY"

Relationships

- From DeploymentManifestType: typed association via platform; diagram multiplicity 0..1.
- From Platform: typed association via engine; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
"DOCKER"
```

Runtime

Groups the deployment alternatives for one infrastructure platform inside a DOT.

Choose a deployment from this group rather than treating every entry as a workload to start. Deployment IDs must be distinct within the runtime, and every manifest must target the runtime platform. This declaration is not a running instance or a readiness report.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`platform`	Platform	Required	The infrastructure on which these deployment alternatives can run.
`deployments`	array of Deployment	Required	Named ways to deploy this DOT on the selected platform. minimum items: 1

Relationships

- From Dot: containment via runtimes; diagram multiplicity 0..*.
- To Deployment: containment via deployments; diagram multiplicity 1..*.
- To Platform: containment via platform; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "platform": {
    "engine": "DOCKER",
    "version": "latest"
  },
  "deployments": [
    {
      "id": "image",
      "manifest": {
        "type": "DOCKER_IMAGE",
        "image": "docker.io/example/api:1.0.0"
      },
      "services": []
    }
  ]
}
```

ServicePort

Names a service endpoint port so interfaces can bind to it without copying runtime addresses.

Port names must be distinct within the service and must match AnchorBinding.port exactly. A protocol label describes the endpoint; it does not itself supply an API Contract or authorize a probe. Host forwarding and externally reachable addresses are deployment-instance concerns.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`name`	string	Required	The logical port name used by bindings and references. minimum length: 1
`protocol`	string	Required	The application or transport protocol label for this endpoint. minimum length: 1
`port`	integer	Required	The numeric port exposed by the service inside its deployment. minimum: 1; maximum: 65535

Relationships

- From DeploymentService: containment via ports; diagram multiplicity 0..*.
- From AnchorBinding: ID/name reference via port.

Java compatibility shape example - not runnable

```
{
  "name": "http",
  "protocol": "TCP",
  "port": 8080
}
```

Vagrantfile

Embeds the complete Vagrantfile used to provision a virtual-machine deployment.

The payload is Ruby DSL, not a separately structured box declaration or an adjacent-file lookup. Review its provisioning, mounts and networking before execution. Service configuration and instance overrides remain separate; merely validating the declaration does not run or certify the virtual machine.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`payload`	string	Required	The complete Vagrantfile program supplied to Vagrant. minimum length: 1
`type`	DeploymentManifestType	Required	The discriminator selecting the Vagrant deployment. exact value "VAGRANTFILE"

Relationships

- From DeploymentManifest: inheritance.

Java compatibility shape example - not runnable

```
{
  "type": "VAGRANTFILE",
  "payload": "Vagrant.configure('2') {}"
}
```

VolumeMount

Declares a logical storage mount expected by a service.

Keep the logical name and workload destination separate from a host bind path, named volume or persistent-volume claim. The instance supplies the concrete source and authorization. A declared read-only expectation must be enforceable by the selected adapter; it is not a storage allocation or a cleanup policy.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`description`	string	Conditional / optional	An explanation of the expected contents or use of the mount.
`name`	string	Required	The logical identifier used to bind storage to this mount. minimum length: 1
`path`	string	Required	The destination path inside the workload. minimum length: 1
`readOnly`	boolean	Conditional / optional	Whether the workload expects read-only access to this mount.
`required`	boolean	Conditional / optional	Whether the deployer must provide a concrete storage source.

Relationships

- From DeploymentConfig: containment via volumes; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "name": "data",
  "path": "/var/lib/example",
  "readOnly": false
}
```

WebAssemblyExecution

Selects the WASI execution world expected by a WebAssembly artifact.

Match the artifact's implemented world to the intended command or HTTP service behavior. Command arguments and HTTP service ports have different executor rules. This selector is not a generic promise to run every WebAssembly component or provide unrestricted operating-system access.

Java authority / Generated JSON Schema

Exact JSON values: `WASI_CLI`, `WASI_HTTP`.

JSON representation: string. allowed: "WASI_CLI", "WASI_HTTP"

Relationships

- From WebAssemblyModule: typed association via execution; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
"WASI_CLI"
```

WebAssemblyModule

References exact WebAssembly bytes and the WASI execution world used to run them.

Keep the module location and checksum together so the adapter can verify bytes before execution. The standard adapter supports local/file and HTTPS locations, and command arguments apply only to CLI execution. Service-declared configuration, explicit directory preopens and adapter support determine available resources; choosing a world does not grant arbitrary host access.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`arguments`	array of string	Required	Arguments passed to the CLI command after the module path.
`execution`	WebAssemblyExecution	Required	The WASI world used to invoke the artifact. allowed: "WASI_CLI", "WASI_HTTP"
`module`	string	Required	The location from which the adapter obtains the module or component. minimum length: 1; pattern restrictions (see schema)
`sha256`	string	Required	The lower-case SHA-256 pin of the exact artifact bytes. minimum length: 1; pattern restrictions (see schema)
`type`	DeploymentManifestType	Required	The discriminator selecting the WebAssembly deployment. exact value "WASM_MODULE"

Relationships

- From DeploymentManifest: inheritance.
- To WebAssemblyExecution: typed association via execution; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "type": "WASM_MODULE",
  "module": "module.wasm",
  "execution": "WASI_CLI",
  "sha256": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
  "arguments": []
}
```

Compositions and connections

AnchorReference

Identify a logical Anchor on one composition member.

The Anchor ID belongs to the referenced DOT, while the instance name belongs to this composition. Neither field is an address, a service name or a Contract operation selector; the executor resolves those relationships from the selected definition.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`instance`	string	Required	Name of the composition member containing the Anchor. minimum length: 1
`anchorId`	string	Required	Exact DOT-local Anchor key on that member. minimum length: 1; pattern restrictions (see schema); conditional constraints (see schema)

Relationships

- From ExposedAnchor: containment via target; diagram multiplicity 1.
- From Wire: containment via from; diagram multiplicity 1.
- From Wire: containment via to; diagram multiplicity 1.
- To DotInstance: ID/name reference via instance.
- To Anchor: ID/name reference via anchorId.

Java compatibility shape example - not runnable

```
{
  "instance": "metrics",
  "anchorId": "http-api"
}
```

CdiDeviceBinding

Assign an installed CDI device to a logical device requirement of a deployment.

Bind the declared requirement to a concrete device of the same CDI kind. This does not install a CDI specification, allocate hardware or authorize an arbitrary host device path; availability and supported adapter behavior remain deployment concerns.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`requirement`	string	Required	Logical device-requirement name declared by the selected service configuration. minimum length: 1
`device`	string	Required	Fully qualified CDI device identity chosen by the operator. maximum length: 381; pattern restrictions (see schema)

Relationships

- From DeploymentInputs: containment via cdiDevices; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "requirement": "gpu",
  "device": "vendor.example/gpu=0"
}
```

ComposeWorkloadBinding

Select the Compose service implementing one logical DOT service.

Use the service key in the declared Compose manifest, not a generated container name or the logical DOT service ID by assumption. The executor verifies that the mapping is valid for the selected deployment.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`service`	string	Required	Compose manifest service key selected for this logical service. minimum length: 1

Relationships

- From ServiceWorkloadBinding: containment via compose; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "service": "api"
}
```

Composition

Describe one reusable use case by selecting DOT instances and connecting their logical interfaces.

This is a separate aggregate, not a DOT containing other DOT definitions. Instance references must name declared members and explicit readiness dependencies must be acyclic; interface wires may form cycles. Catalogue resolution, reviewed locks, credentials, deployment ownership and live acceptance remain separate executor concerns.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`apiVersion`	string	Required	Identifies the portable composition input profile understood by the reader. exact value "dot.mare6g.eu/v1"; pattern restrictions (see schema)
`kind`	string	Required	Distinguishes a composition declaration from other document kinds. exact value "Composition"; pattern restrictions (see schema)
`id`	string	Required	Stable identity used to associate lifecycle ownership with this composition, independent of its display name. pattern restrictions (see schema)
`name`	string	Required	Human-readable name for the use case, not a replacement for its ownership identity. minimum length: 1
`version`	string	Required	Version label for this use-case declaration; it is not the version of every participating tool. minimum length: 1

JSON field	Value	Base presence	Meaning / constraints
`dots`	array of DotInstance	Required	Named instances selecting canonical DOT definitions and their deployments. minimum items: 1
`contracts`	array of ContractInput	Conditional / optional	Explicit interface-document inputs resolved from authoritative owners, without transferring Contract ownership.
`wires`	array of Wire	Conditional / optional	Logical connections from provided Anchors to required Anchors.
`exposedAnchors`	array of ExposedAnchor	Conditional / optional	Names for selected member Anchors at the composition boundary; these do not themselves publish a network listener.

Relationships

- To ContractInput: containment via contracts; diagram multiplicity 0..*.
- To DotInstance: containment via dots; diagram multiplicity 1..*.
- To ExposedAnchor: containment via exposedAnchors; diagram multiplicity 0..*.
- To Wire: containment via wires; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "apiVersion": "dot.mare6g.eu/v1",
  "kind": "Composition",
  "id": "00000000-0000-4000-8000-000000000001",
  "name": "example",
  "version": "v1",
  "dots": [
    {
      "name": "metrics",
      "source": "repository://Example@1.0.0",
      "dotId": "Example@1.0.0",
      "dotVersion": "1.0.0"
    }
  ]
}
```

ContractInput

Make an authoritative Contract document available to composition resolution.

Reference the originating DOT instead of copying its Contract declaration or payload into a consumer. The owner supplies the pinned URL and digest. Historical wrapper sources remain a compatibility path, not a fallback when an owning declaration is missing.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`id`	string	Required	Exact originating Contract identity used by the selected interfaces. minimum length: 1; pattern restrictions (see schema)
`source`	string	Required	Source identifying the owning DOT whose root declaration supplies the Contract metadata and verified payload. minimum length: 1

Relationships

- From Composition: containment via contracts; diagram multiplicity 0..*.
- To Contract: ID/name reference via id.

Canonical Prometheus projection

```
{
  "id": "Prometheus@3.15.1::OpenAPI::HTTP",
  "source": "repository://Prometheus@3.15.1"
}
```

DeploymentConfigFile

Bind a local configuration file to a service-declared file destination.

The source is deployment input, not another file the executor may inspect for deployment instructions. The target must match the selected service configuration; source existence, path resolution and unambiguous service selection are executor checks.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`service`	string	Conditional / optional	Logical deployment-service ID receiving the file. minimum length: 1
`source`	string	Required	Local input file to read, resolved in the composition input context. minimum length: 1
`target`	string	Required	File path declared by the service where this configuration is supplied. minimum length: 1

Relationships

- From DeploymentInputs: containment via configFiles; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "service": "api",
  "source": "inputs/config.yaml",
  "target": "/etc/example/config.yaml"
}
```

DeploymentEnvironment

Provide one declared environment setting to a selected deployment service.

Choose either a literal value or a secret reference, never both. An empty literal remains a deliberate value. Sensitive settings use references rather than authored secrets; omitted service selection is usable only when the executor can resolve it unambiguously.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`service`	string	Conditional / optional	Logical deployment-service ID receiving this setting. minimum length: 1
`name`	string	Required	Environment-variable name declared by that service. minimum length: 1
`value`	string	Conditional / optional	Literal configuration text for a non-sensitive setting.
`secretRef`	SecretReference	Conditional / optional	Reference to the private value supplied through an executor-supported secret store.

Relationships

- To SecretReference: containment via secretRef; diagram multiplicity 0..1.
- From DeploymentInputs: containment via environment; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "name": "MODE",
  "value": ""
}
```

DeploymentInputs

Supply deployment-instance choices without changing the reusable DOT definition.

Inputs target configuration declared by services of the selected deployment. They do not introduce another manifest or imply permission to use undeclared environment settings, workloads, host resources or hardware. Resolution and adapter support are checked by the executor.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`environment`	array of DeploymentEnvironment	Conditional / optional	Service-scoped environment values or references to private values.
`configFiles`	array of DeploymentConfigFile	Conditional / optional	Local configuration files bound to declared service file destinations.
`volumes`	array of DeploymentVolumeSource	Conditional / optional	Explicit host storage sources bound to declared logical volume requirements.
`workloads`	array of ServiceWorkloadBinding	Conditional / optional	Mappings from logical service IDs to actual Compose or Kubernetes workloads.
`ociRuntime`	OciRuntimeGrant	Conditional / optional	Explicit authorization for the declared Linux capability and host-namespace requirements. conditional constraints (see schema)
`cdiDevices`	array of CdiDeviceBinding	Conditional / optional	Concrete CDI device assignments for declared logical device requirements. distinct items

Relationships

- To CdiDeviceBinding: containment via cdiDevices; diagram multiplicity 0..*.
- To DeploymentConfigFile: containment via configFiles; diagram multiplicity 0..*.
- To DeploymentEnvironment: containment via environment; diagram multiplicity 0..*.
- To DeploymentVolumeSource: containment via volumes; diagram multiplicity 0..*.
- To OciRuntimeGrant: containment via ociRuntime; diagram multiplicity 0..1.
- To ServiceWorkloadBinding: containment via workloads; diagram multiplicity 0..*.
- From DotInstance: containment via deploymentInputs; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "environment": [
    {
      "service": "api",
      "name": "MODE",
      "value": "demo"
    }
  ]
}
```

DeploymentVolumeSource

Bind an explicit host storage source to a declared service volume.

This records an operator grant, not a request to discover or create arbitrary host storage. The current executor requires a supported direct-container adapter and an absolute bind source; read-only intent comes from the declared volume requirement rather than an invented field here.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`service`	string	Conditional / optional	Logical deployment-service ID whose volume is being bound. minimum length: 1
`name`	string	Required	Logical volume name declared by that service, not the host path. minimum length: 1
`type`	string	Required	Storage-source mechanism interpreted by the supported runtime adapter. exact value "bind"; pattern restrictions (see schema)
`source`	string	Required	Explicit host location supplied for the declared volume. minimum length: 1

Relationships

- From DeploymentInputs: containment via volumes; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "service": "api",
  "name": "data",
  "type": "bind",
  "source": "./data"
}
```

DotInstance

Select one canonical DOT release for a named role in a composition.

Use the instance name in graph references and the DOT identity for catalogue resolution. A qualified source must agree with the selected DOT metadata. Choose a deployment ID rather than a manifest-type label; dependencies express readiness order, not interface compatibility or application data flow.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`name`	string	Required	Composition-local name used by wires, dependencies and other instance references. minimum length: 1
`source`	string	Required	Location or qualified catalogue reference from which the canonical definition is resolved. minimum length: 1
`dotId`	string	Required	Exact identity expected in the resolved DOT definition. minimum length: 1; pattern restrictions (see schema)
`dotVersion`	string	Required	Exact component version expected in that definition, checked independently of the source location. minimum length: 1
`deployment`	string	Conditional / optional	ID of the deployment alternative to use from the selected DOT. minimum length: 1
`dependsOn`	array of string	Conditional / optional	Other instance names whose readiness is a prerequisite for this instance. distinct items
`deploymentInputs`	DeploymentInputs	Conditional / optional	Configuration, workload selection and explicit runtime grants for this deployment instance.

Relationships

- From Composition: containment via dots; diagram multiplicity 1..*.
- To DeploymentInputs: containment via deploymentInputs; diagram multiplicity 0..1.
- To Dot: ID/name reference via dotId / source.
- From AnchorReference: ID/name reference via instance.

Java compatibility shape example - not runnable

```
{
  "name": "metrics",
  "source": "repository://Example@1.0.0",
  "dotId": "Example@1.0.0",
  "dotVersion": "1.0.0",
  "dependsOn": [
    "identity"
  ]
}
```

ExposedAnchor

Give a member Anchor a stable name at the composition boundary.

Exposure names an existing logical interface; it does not create an API, a public listener or an authorization policy. Forwarding, reachable addresses and live checks are explicit executor operations.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`name`	string	Required	Composition-local boundary name used to refer to this exposed interface. minimum length: 1
`target`	AnchorReference	Required	Member Anchor selected as the boundary interface.

Relationships

- From Composition: containment via exposedAnchors; diagram multiplicity 0..*.
- To AnchorReference: containment via target; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "name": "api",
  "target": {
    "instance": "metrics",
    "anchorId": "http-api"
  }
}
```

KubernetesWorkloadBinding

Select the Kubernetes workload implementing one logical DOT service, including Helm-rendered workloads.

Labels must identify the intended workload unambiguously within the deployment scope. Select a container when needed rather than relying on order. This is not a grant to adopt unrelated resources or a replacement for lifecycle ownership checks.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`matchLabels`	map of string	Required	Label key/value pairs used to select the workload. minimum keys: 1
`container`	string	Conditional / optional	Name of the container within the selected workload to which service configuration applies. minimum length: 1

Relationships

- From ServiceWorkloadBinding: containment via kubernetes; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "matchLabels": {
    "app": "example"
  },
  "container": "api"
}
```

Materializer

Turn a resolved wire value or local input file into declared consumer configuration.

A file destination can copy a source file or inject a resolved value at a JSON Pointer; injection couples valueFrom with that pointer. An environment destination names a declared variable and has no source file. URI templates transform the selected value, not arbitrary runtime state. These actions configure a connection; they do not translate protocols.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`type`	string	Required	Destination mechanism used to supply configuration to the consumer. allowed: "CONFIG_FILE", "ENVIRONMENT_VARIABLE"
`source`	string	Conditional / optional	Local source file for a file-based materialization. minimum length: 1

JSON field	Value	Base presence	Meaning / constraints
`valueFrom`	string	Conditional / optional	Which resolved endpoint, operation or channel value to use for injection. allowed: "ENDPOINT_URL", "OPERATION_URL", "ENDPOINT_HOST", "ENDPOINT_HOST_PORT", "ENDPOINT_PORT", "CHANNEL_ADDRESS"
`uriTemplate`	string	Conditional / optional	URI template applied to the selected value through its value variable.
`target`	MaterializerTarget	Required	Exact instance, service and declared configuration destination receiving the result.

Relationships

- To MaterializerTarget: containment via target; diagram multiplicity 1.
- To MaterializerType: typed association via type; diagram multiplicity 1.
- To MaterializerValueFrom: typed association via valueFrom; diagram multiplicity 0..1.
- From Wire: containment via materializer; diagram multiplicity 0..1.
- From Wire: containment via materializers; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "type": "ENVIRONMENT_VARIABLE",
  "valueFrom": "ENDPOINT_URL",
  "target": {
    "instance": "client",
    "service": "api",
    "environmentVariable": "API_URL"
  }
}
```

MaterializerTarget

Identify the declared consumer configuration location written by a materializer.

Select fields matching the materializer destination rather than mixing file and environment targets. File paths and variable names must already be declared by the target service. The JSON Pointer addresses content inside the source configuration, not a filesystem path or a schema field.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`instance`	string	Required	Composition member receiving the materialized configuration. minimum length: 1
`service`	string	Required	Logical deployment service receiving the configuration. minimum length: 1
`path`	string	Conditional / optional	Declared service file destination used for file materialization. minimum length: 1
`jsonPointer`	string	Conditional / optional	Location in structured configuration where a resolved value is injected. pattern restrictions (see schema)
`environmentVariable`	string	Conditional / optional	Declared service environment-variable name receiving the resolved value. minimum length: 1

Relationships

- From Materializer: containment via target; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "instance": "client",
  "service": "api",
  "path": "config.json",
  "jsonPointer": "/routes/0/uri"
}
```

MaterializerType

Choose the consumer configuration destination for a materializer.

The choice determines which source and target fields make sense together. It does not select a deployment mechanism or grant permission to modify undeclared consumer settings. Use the exact schema spelling.

Java authority / Generated JSON Schema

Exact JSON values: `CONFIG_FILE`, `ENVIRONMENT_VARIABLE`.

JSON representation: string. allowed: "CONFIG_FILE", "ENVIRONMENT_VARIABLE"

Relationships

- From Materializer: typed association via type; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
"CONFIG_FILE"
```

MaterializerValueFrom

Choose which resolved connection value a materializer uses.

Endpoint, operation and channel values have different meanings; select the one expected by the consumer. This is not a general expression language, a secret lookup or a source of live runtime state. The schema owns the supported choices.

Java authority / Generated JSON Schema

Exact JSON values: `ENDPOINT_URL`, `OPERATION_URL`, `ENDPOINT_HOST`, `ENDPOINT_HOST_PORT`, `ENDPOINT_PORT`, `CHANNEL_ADDRESS`.

JSON representation: string. allowed: "ENDPOINT_URL", "OPERATION_URL", "ENDPOINT_HOST", "ENDPOINT_HOST_PORT", "ENDPOINT_PORT", "CHANNEL_ADDRESS"

Relationships

- From Materializer: typed association via valueFrom; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
"ENDPOINT_URL"
```

NetworkBinding

Declare an existing gateway through which a wire crosses runtime boundaries.

The operator supplies reachability; the executor does not provision the gateway, change host routing or turn a logical wire into connectivity. The declaration is separate from active reachability and Contract checks.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`adapter`	string	Required	Cross-runtime connection strategy understood by the executor. exact value "EXTERNAL_GATEWAY"; pattern restrictions (see schema)
`host`	string	Required	Gateway host reachable from the consuming runtime, without a URL scheme or path. minimum length: 1; maximum length: 253; pattern restrictions (see schema)
`port`	integer	Required	Gateway listener port corresponding to the selected endpoint. minimum: 1; maximum: 65535

Relationships

- From Wire: containment via network; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "adapter": "EXTERNAL_GATEWAY",
  "host": "gateway.example.org",
  "port": 443
}
```

OciRuntimeGrant

Authorize the Linux runtime privileges explicitly declared by a selected deployment.

A declaration of elevated requirements is not permission to grant them. The executor checks the exact grant against the DOT requirements and rejects unsupported adapters or mismatches; this is not an unrestricted privileged-container switch.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`capabilities`	array of string	Conditional / optional	Linux capabilities the operator explicitly permits for the selected workload. distinct items
`hostNamespaces`	array of string	Conditional / optional	Host namespaces whose inheritance the operator explicitly permits. distinct items

Relationships

- From DeploymentInputs: containment via ociRuntime; diagram multiplicity 0..1.
- To OciLinuxNamespace: typed association via hostNamespaces; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "capabilities": [
    "CAP_NET_RAW"
  ]
}
```

OperationParameterBinding

Supply one named operation or channel parameter on a wire.

The containing map supplies the parameter name; this object supplies its value. The executor checks it against the selected Contract operation. Do not use operation parameters as a substitute for secret references or undeclared deployment configuration.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`value`	string	Required	Text used to bind the named parameter while resolving the operation or channel. minimum length: 1

Relationships

- From Wire: containment via parameters; diagram multiplicity 0..*.

Java compatibility shape example - not runnable

```
{
  "value": "demo"
}
```

SecretReference

Identify a private value without storing that value in the composition.

A reference is not the credential itself and does not imply that every store or adapter is supported. The executor applies store-specific lookup and native-reference rules; keep resolved values out of authored documents, locks and public evidence.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`store`	string	Required	Secret-store mechanism used to interpret the reference. minimum length: 1
`name`	string	Required	Secret object or private input name within that store. minimum length: 1
`key`	string	Conditional / optional	Entry within the named secret where the selected store needs one.
`version`	string	Conditional / optional	Store-specific secret revision selector, not a DOT or composition version.

Relationships

- From AsyncAPIProbeAuthentication: containment via passwordSecretRef; diagram multiplicity 1.
- From DeploymentEnvironment: containment via secretRef; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "store": "env",
  "name": "DEMO_PASSWORD"
}
```

SecurityBinding

Explain how one required Contract security scheme is supplied by a consuming service.

The executor checks the scheme against the selected operation and the mechanism against its supported security profile. Credential parameters refer to declared configuration on the consuming instance, not a new identity provider or literal token. Declared scopes do not by themselves prove that access was granted.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`scheme`	string	Required	Security-scheme name declared by the selected Contract operation. minimum length: 1
`mechanism`	string	Required	Executor-supported authentication mechanism used to satisfy that scheme. minimum length: 1
`participant`	ServiceReference	Required	Consuming instance and logical service whose declared inputs supply the credentials.
`parameters`	map of SecurityParameterBinding	Required	Mechanism-specific parameter names mapped to declared service environment settings. minimum keys: 1
`scopes`	array of string	Conditional / optional	Requested scope names used when satisfying the Contract security requirement. distinct items

Relationships

- To SecurityParameterBinding: containment via parameters; diagram multiplicity 1..*.
- To ServiceReference: containment via participant; diagram multiplicity 1.
- From WireSecurity: containment via bindings; diagram multiplicity 1..*.

Java compatibility shape example - not runnable

```
{
  "scheme": "bearer",
  "mechanism": "HTTP_BEARER",
  "participant": {
    "instance": "client",
    "service": "api"
  },
  "parameters": {
    "token": {
      "environmentVariable": "DEMO_TOKEN"
    }
  },
  "scopes": [
    "read"
  ]
}
```

SecurityParameterBinding

Connect one authentication parameter to a declared service environment setting.

The containing security binding determines what the parameter means. This object identifies where its value is configured; sensitive values must still be supplied through the proper private input path rather than embedded here.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`environmentVariable`	string	Required	Declared variable on the participant service supplying this authentication parameter. minimum length: 1

Relationships

- From SecurityBinding: containment via parameters; diagram multiplicity 1..*.

Java compatibility shape example - not runnable

```
{
  "environmentVariable": "DEMO_TOKEN"
}
```

ServicePortReference

Select a declared service port used as a concrete wire endpoint.

The port field is a named service-port key, not a numeric port or URL. Endpoint selection may identify broker transport separately from the logical message producer; the executor checks the selected endpoint against the wire and deployment.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`instance`	string	Required	Composition member supplying the endpoint service. minimum length: 1
`service`	string	Required	Logical deployment service supplying the endpoint. minimum length: 1
`port`	string	Required	Declared port name on that service. minimum length: 1

Relationships

- From Wire: containment via endpoint; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "instance": "metrics",
  "service": "api",
  "port": "http"
}
```

ServiceReference

Identify a logical service on a composition member.

Service identities come from the selected deployment, not from runtime container or Pod names. Java checks graph membership; the executor additionally resolves the service against the chosen DOT deployment.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`instance`	string	Required	Composition member containing the service. minimum length: 1

JSON field	Value	Base presence	Meaning / constraints
`service`	string	Required	Logical service ID within that member deployment. minimum length: 1

Relationships

- From SecurityBinding: containment via participant; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "instance": "metrics",
  "service": "api"
}
```

ServiceWorkloadBinding

Map a logical deployment service to the workload that actually materializes it.

Select one workload mechanism for the deployment in use. The outer service is a DOT service ID; the nested selector names runtime objects. This distinction is essential for multi-service manifests and prevents the executor from guessing or operating on an unrelated workload.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`service`	string	Required	Logical service ID in the selected deployment. minimum length: 1
`compose`	ComposeWorkloadBinding	Conditional / optional	Selector for the Compose service implementing that logical service.
`kubernetes`	KubernetesWorkloadBinding	Conditional / optional	Selector for the Kubernetes workload and, where necessary, its container.

Relationships

- From DeploymentInputs: containment via workloads; diagram multiplicity 0..*.
- To ComposeWorkloadBinding: containment via compose; diagram multiplicity 0..1.
- To KubernetesWorkloadBinding: containment via kubernetes; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "service": "api",
  "compose": {
    "service": "server"
  }
}
```

Wire

Connect a provided Anchor to a required Anchor and declare how the consumer receives its connection details.

The arrow expresses logical provision, not necessarily packet direction. Parameter, endpoint, security and materializer choices must agree with the resolved interfaces. Use the singular or plural materializer form, not both. A compatible declaration or configured connection is not proof of network reachability or a successful live operation.

JSON field	Value	Base presence	Meaning / constraints
`name`	string	Required	Composition-local name used to identify this connection and select its probe. minimum length: 1
`from`	AnchorReference	Required	Provider-side Anchor reference.
`to`	AnchorReference	Required	Consumer-side Anchor reference.
`parameters`	map of OperationParameterBinding	Conditional / optional	Named values used to bind parameters of the selected operation or channel. minimum keys: 1
`endpoint`	ServicePortReference	Conditional / optional	Explicit service-port endpoint when the wire needs one instead of the inferred binding.
`network`	NetworkBinding	Conditional / optional	Externally supplied gateway address for a cross-runtime connection.
`materializer`	Materializer	Conditional / optional	Single consumer-configuration action for this wire. conditional constraints (see schema)
`materializers`	array of Materializer	Conditional / optional	Several consumer-configuration actions derived from this same wire. minimum items: 1
`security`	WireSecurity	Conditional / optional	Bindings connecting Contract security requirements to declared consumer credential inputs.
`probe`	WireProbe	Conditional / optional	Optional protocol-specific fixture for a supported active Contract check. minimum keys: 1

Relationships

- From Composition: containment via wires; diagram multiplicity 0..*.
- To AnchorReference: containment via from; diagram multiplicity 1.
- To AnchorReference: containment via to; diagram multiplicity 1.
- To Materializer: containment via materializer; diagram multiplicity 0..1.
- To Materializer: containment via materializers; diagram multiplicity 0..*.
- To NetworkBinding: containment via network; diagram multiplicity 0..1.
- To OperationParameterBinding: containment via parameters; diagram multiplicity 0..*.
- To ServicePortReference: containment via endpoint; diagram multiplicity 0..1.
- To WireProbe: containment via probe; diagram multiplicity 0..1.
- To WireSecurity: containment via security; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "name": "api",
  "from": {
    "instance": "provider",
    "anchorId": "http-api"
  },
  "to": {
    "instance": "consumer",
    "anchorId": "http-api"
  }
}
```

WireSecurity

Bind security requirements of the selected Contract operation to consumer inputs.

Bindings must satisfy a supported complete security requirement alternative, not merely name a familiar authentication technology. Resolution checks declarations; successful authentication, authorization and policy behavior require separate live evidence.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`bindings`	array of SecurityBinding	Required	Security-scheme bindings participating in this wire authentication configuration. minimum items: 1

Relationships

- To SecurityBinding: containment via bindings; diagram multiplicity 1..*.
- From Wire: containment via security; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "bindings": [
    {
      "scheme": "bearer",
      "mechanism": "HTTP_BEARER",
      "participant": {
        "instance": "client",
        "service": "api"
      },
      "parameters": {
        "token": {
          "environmentVariable": "DEMO_TOKEN"
        }
      }
    }
  ]
}
```

Probes and assertions

AsyncAPIProbe

Describe an active broker delivery check tied to the selected AsyncAPI operation and message.

The executor verifies the message against the Contract and injects fresh correlation at the selected pointer before testing delivery. Match the transport to the resolved broker bindings; stale or unrelated messages are not successful acceptance. This does not establish broker durability, production authorization or end-to-end business processing.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`protocol`	string	Required	Broker transport used to execute this delivery fixture. allowed: "MQTT", "NATS", "KAFKA", "AMQP"
`message`	object	Required	JSON message template whose application content is checked against the selected Contract message.
`correlationPointer`	string	Required	Location within the message where the fresh probe correlation value is inserted and later checked. pattern restrictions (see schema)
`qos`	integer	Conditional / optional	Requested MQTT delivery quality for the message exchange. minimum: 0; maximum: 2
`authentication`	AsyncAPIProbeAuthentication	Conditional / optional	Broker login configuration for a supported authenticated probe.

Relationships

- To AsyncAPIProbeAuthentication: containment via authentication; diagram multiplicity 0..1.
- To AsyncAPIProtocol: typed association via protocol; diagram multiplicity 1.
- From WireProbe: containment via asyncapi; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "protocol": "MQTT",
  "message": {
    "id": "demo"
  },
  "correlationPointer": "/id",
  "qos": 1
}
```

AsyncAPIProbeAuthentication

Provide broker login identity and a reference to its private password for an active delivery check.

The password stays in an executor-supported secret source rather than in the fixture. Whether authentication is supported and required depends on the selected transport and Contract bindings; this object does not configure the broker itself.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`username`	string	Required	Login identity presented by the probe client. minimum length: 1

JSON field	Value	Base presence	Meaning / constraints
`passwordSecretRef`	SecretReference	Required	Reference used to obtain the private broker password at execution time.

Relationships

- To SecretReference: containment via passwordSecretRef; diagram multiplicity 1.
- From AsyncAPIProbe: containment via authentication; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "username": "demo",
  "passwordSecretRef": {
    "store": "env",
    "name": "DEMO_PASSWORD"
  }
}
```

AsyncAPIProtocol

Select the broker transport used by an AsyncAPI active probe.

This choice is separate from message ownership and must agree with the resolved transport binding and supported adapter. It does not declare that every feature of that broker protocol is implemented. Use the exact values emitted by the schema.

Java authority / Generated JSON Schema

Exact JSON values: `MQTT`, `NATS`, `KAFKA`, `AMQP`.

JSON representation: string. allowed: "MQTT", "NATS", "KAFKA", "AMQP"

Relationships

- From AsyncAPIProbe: typed association via protocol; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
"MQTT"
```

GRPCProbe

Describe a reflection-based RPC check with a finite request and response sequence.

Choose the single-request or request-sequence form, not both, to match the selected RPC shape. The current profile relies on server reflection. Finite stream completion, response count and content assertions are separate checks; declaring a fixture is not proof that a server implements the RPC.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`plaintext`	boolean	Conditional / optional	Whether the probe uses an unencrypted connection instead of its TLS transport.
`serverReflection`	boolean	Required	Declaration that server reflection is used to obtain the executable RPC description. exact value true
`request`	object	Conditional / optional	Single JSON request message for an RPC with a non-streaming request side.

JSON field	Value	Base presence	Meaning / constraints
`requests`	array of object	Conditional / optional	Ordered JSON messages supplied to a client-streaming request side. minimum items: 1
`responseCount`	integer	Conditional / optional	Expected number of responses after the finite RPC completes. minimum: 1
`responseMatch`	string	Conditional / optional	Which received responses must satisfy the content expectation. allowed: "ALL", "ANY", "FIRST", "LAST"
`expect`	JSONExpectation	Conditional / optional	JSON value assertion evaluated against the selected response messages.

Relationships

- To GRPCResponseMatch: typed association via responseMatch; diagram multiplicity 0..1.
- To JSONExpectation: containment via expect; diagram multiplicity 0..1.
- From WireProbe: containment via grpc; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "serverReflection": true,
  "request": {}
}
```

GRPCResponseMatch

Choose which responses in a finite RPC result must match an expectation.

Response selection does not replace the expected response count or the RPC completion check. Choose a policy appropriate to the operation instead of treating one matching streaming message as proof of the entire exchange. The schema owns the policy names.

Java authority / Generated JSON Schema

Exact JSON values: `ALL`, `ANY`, `FIRST`, `LAST`.

JSON representation: string. allowed: "ALL", "ANY", "FIRST", "LAST"

Relationships

- From GRPCProbe: typed association via responseMatch; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
"ALL"
```

GraphQLOperation

Supply a supporting GraphQL operation within a probe sequence.

Use this for the verification, trigger or cleanup role selected by GraphQLProbe. The executor checks the document against the same verified schema and the operation kind required for that role; an operation name is not a substitute for the document.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`document`	string	Required	Executable GraphQL document for the supporting operation. minimum length: 1
`operationName`	string	Conditional / optional	Name selecting the intended operation within that document. pattern restrictions (see schema)
`variables`	object	Conditional / optional	Named GraphQL inputs for this supporting operation, separate from the main probe inputs.

Relationships

- From GraphQLProbe: containment via cleanup; diagram multiplicity 0..1.
- From GraphQLProbe: containment via trigger; diagram multiplicity 0..1.
- From GraphQLProbe: containment via verify; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "document": "query Status { status }",
  "operationName": "Status",
  "variables": {}
}
```

GraphQLProbe

Describe a GraphQL query, mutation or subscription check against the selected interface.

The executable document must agree with the verified schema and selected operation. The current executor requires a verification query and cleanup mutation for mutation probes, and trigger and cleanup mutations for subscription probes; the transport must suit the operation. These are bounded acceptance fixtures, not a general GraphQL client or a guarantee of application rollback.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`path`	string	Required	Absolute HTTP endpoint path on the resolved service, starting with a slash; not a full URL. pattern restrictions (see schema)
`document`	string	Required	Executable GraphQL document used for the main check. minimum length: 1
`operationName`	string	Conditional / optional	Operation name selecting the intended operation within that document. pattern restrictions (see schema)
`variables`	object	Conditional / optional	Named GraphQL input values for the main operation.
`transport`	string	Conditional / optional	Delivery transport used for the main operation. allowed: "HTTP", "SSE"
`expect`	JSONExpectation	Conditional / optional	JSON result assertion for the supported probe sequence.
`verify`	GraphQLOperation	Conditional / optional	Follow-up query used to verify the effect of a mutation.
`trigger`	GraphQLOperation	Conditional / optional	Mutation used to produce an event for a subscription check.
`cleanup`	GraphQLOperation	Conditional / optional	Mutation intended to remove the test effect after the supported probe sequence.

Relationships

- To GraphQLOperation: containment via cleanup; diagram multiplicity 0..1.
- To GraphQLOperation: containment via trigger; diagram multiplicity 0..1.
- To GraphQLOperation: containment via verify; diagram multiplicity 0..1.
- To GraphQLTransport: typed association via transport; diagram multiplicity 0..1.
- To JSONExpectation: containment via expect; diagram multiplicity 0..1.
- From WireProbe: containment via graphql; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "path": "/graphql",
  "document": "{ status }",
  "transport": "HTTP"
}
```

GraphQLTransport

Choose how the GraphQL probe operation is delivered.

Transport and operation kind are separate choices and must form an executor-supported combination. Selecting a streaming transport does not make a query a subscription or grant support for arbitrary GraphQL protocols. Use the exact schema spelling.

Java authority / Generated JSON Schema

Exact JSON values: `HTTP`, `SSE`.

JSON representation: string. allowed: "HTTP", "SSE"

Relationships

- From GraphQLProbe: typed association via transport; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
"HTTP"
```

JSONExpectation

Assert that a selected part of a JSON result equals an expected value.

The pointer selects content, not a URL or query expression; an empty pointer selects the whole document. Explicit null is an expected value and must not be confused with an omitted equals field. Protocol-specific execution determines which result document is checked.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`pointer`	string	Required	JSON Pointer locating the value to compare within the result document. pattern restrictions (see schema)
`equals`	JSON value	Required	Expected JSON value, including an explicit null when that is the intended assertion.

Relationships

- From GRPCProbe: containment via expect; diagram multiplicity 0..1.

- From GraphQLProbe: containment via expect; diagram multiplicity 0..1.
- From OpenAPIProbe: containment via expect; diagram multiplicity 1.

Java compatibility shape example - not runnable

```
{
  "pointer": "/status",
  "equals": "ready"
}
```

OpenAPIProbe

Add a JSON response-value assertion to a supported OpenAPI wire probe.

The operation comes from the wire Anchor and verified Contract, not a second handwritten URL or operation definition. A successful HTTP status alone is different from a matched response expectation; executor restrictions still determine which operations can be probed.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`expect`	JSONExpectation	Required	Result-value assertion applied in addition to the native Contract response checks.

Relationships

- To JSONExpectation: containment via expect; diagram multiplicity 1.
- From WireProbe: containment via openapi; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "expect": {
    "pointer": "/status",
    "equals": "ready"
  }
}
```

WireProbe

Choose a protocol-specific active-check fixture for a wire.

Select one protocol fixture matching the resolved Contract standard. Portable fixture validity does not promise that an executor supports every deployment, operation or transport combination. Active checks may call APIs or publish messages and must be deliberately selected; they are not passive topology inspection.

Java authority / Generated JSON Schema

JSON field	Value	Base presence	Meaning / constraints
`openapi`	OpenAPIProbe	Conditional / optional	Response-value expectation for a supported OpenAPI operation.
`graphql`	GraphQLProbe	Conditional / optional	GraphQL operation, transport and any verification or cleanup operations.
`grpc`	GRPCProbe	Conditional / optional	Reflection-based RPC request and finite-response assertions. conditional constraints (see schema)

JSON field	Value	Base presence	Meaning / constraints
`asyncapi`	AsyncAPIProbe	Conditional / optional	Broker delivery fixture with message correlation and optional authentication.

Relationships

- To AsyncAPIProbe: containment via asyncapi; diagram multiplicity 0..1.
- To GRPCProbe: containment via grpc; diagram multiplicity 0..1.
- To GraphQLProbe: containment via graphql; diagram multiplicity 0..1.
- To OpenAPIProbe: containment via openapi; diagram multiplicity 0..1.
- From Wire: containment via probe; diagram multiplicity 0..1.

Java compatibility shape example - not runnable

```
{
  "openapi": {
    "expect": {
      "pointer": "/status",
      "equals": "ready"
    }
  }
}
```